

Database Systems Lab

Data

Christian Rauch

(changed: August 24, 2025)

Lists and Sets

Collections (also called containers) accept elements. In Python, most collections are heterogeneous (i.e., accept different element data types) and mutable (i.e., can be modified after creation).

Lists are indexable and ordered collections.

```
1 my_list = [1, 4, 'b', 4, 5]
2 my_list.append(2)
3 print(my_list)           # [1, 4, 'b', 4, 5, 2]
4 print(my_list[1])        # 4
5 print(my_list[-2])       # 5
6 print(my_list[1:4])      # [4, 'b', 4]
```

Sets contain each element at most once (and guarantee no order).

```
1 my_set = { 2, 3, "a", 4, 5}
2 my_set.add(6)
3 print(my_set)           # {3, 'a', 2, 4, 5, 6}
4 my_set.add(3)           # add an existing element
5 print(my_set)           # {'a', 4, 2, 3, 5, 6}
```

Dictionaries

Dictionaries are unordered collections of key-value pairs. Each key is unique, and keys are used to access the associated values.

Dictionaries are indexed by keys, not by position.

```
1 my_dict = {'a': 1, 'b': 2}
2 my_dict['d'] = 4      # Adding a key-value pair
3
4 print(my_dict)        # {'a': 1, 'b': 2, 'd': 4}
5 print(my_dict['b'])   # 2
6
7 my_dict['a'] = 10     # Updating value for a key
8 print(my_dict)        # {'a': 10, 'b': 2, 'd': 4}
9
10 del my_dict['b']      # Removing a key
11 print(my_dict)       # {'a': 10, 'd': 4}
```

Sorting

Sorting with simple sortable types.

```
1 my_list = [5, 2, 9, 1, 5]
2 my_list.sort()
3 print(my_list)    # [1, 2, 5, 5, 9]
```

Sorting based on a custom key.

```
1 team = [
2     {'name': 'Alice', 'age': 22},
3     {'name': 'Bob', 'age': 20},
4     {'name': 'Charlie', 'age': 23}
5 ]
6
7 team = sorted(team, key=lambda x: x['age'])
8 print(team)
9 # [{'name': 'Bob', 'age': 20},
10 #  {'name': 'Alice', 'age': 22},
11 #  {'name': 'Charlie', 'age': 23}]
```

Projection

For complex types you may want to query specific attributes.

```
1 team = [  
2     {'name': 'Alice', 'age': 22},  
3     {'name': 'Bob', 'age': 20},  
4     {'name': 'Charlie', 'age': 23}  
5 ]  
  
6  
7 names = [s['name'] for s in team]  
8 print(names) # ['Alice', 'Bob', 'Charlie']
```

Projections may involve transformations.

```
1 years_as_adult = [(s['age']-18) for s in team]  
2 print(years_as_adult) # [4, 2, 5]
```

Selection

Selecting elements (filtering based on condition).

```
1 my_list = [1, 4, 6, 7, 9]
2 evens = [x for x in my_list if x % 2 == 0]
3 print(evens) # [4, 6]
```

Alternative approach using filter().

```
1 my_list = [1, 4, 6, 7, 9]
2 evens = list(filter(lambda x: x % 2 == 0,
3                     my_list))
4 print(evens) # [4, 6]
```

Combination

Adding another list (concatenation).

```
1 list1 = [1, 2, 3]
2 list2 = [4, 5, 6]
3 combined = list1 + list2
4 print(combined)    # [1, 2, 3, 4, 5, 6]
```

Intersecting with another list.

```
1 list1 = [1, 2, 3, 4]
2 list2 = [3, 4, 5, 6]
3 intersection = [x for x in list1 if x in list2]
4 print(intersection)    # [3, 4]
```

CSV files

CSV files describe tabular objects.

```
1 x;y;z
2 5;3;7
3 3;2;2
4 9;1;3
```

Read data from CSV files.

```
1 import csv
2 def read_csv(file_name):
3     x = []
4     y = []
5     with open(file_name, 'r') as csvfile:
6         csvreader = csv.reader(csvfile,
7                                 delimiter=',')
8         next(csvreader) # Skip the header
9         for row in csvreader:
10             x.append(float(row[0]))
11             y.append(float(row[1]))
12     return x, y
```


JSON files

JSON files can describe complex objects.

```
1  [  
2      {"x": 1, "y": 2, "z": [9, 7]},  
3      {"x": 2, "y": 4, "z": [5, 3, [1, 2]]},  
4      {"x": 3, "y": 6, "z": [3, "a"]}  
5  ]
```

Read data from JSON files.

```
1  import json  
2  def read_json(file_name):  
3      x = []  
4      y = []  
5      with open(file_name, 'r') as jsonfile:  
6          data = json.load(jsonfile)  
7          for item in data:  
8              x.append(float(item['x']))  
9              y.append(float(item['y']))  
10     return x, y
```

Plotting

Generate plots using matplotlib.

```
1 import matplotlib.pyplot as plt
2
3 def plot_data(x, y):
4     plt.plot(x, y, marker='o', linestyle='-',
5             color='b')
6     plt.title('Example')
7     plt.xlabel('X-axis')
8     plt.ylabel('Y-axis')
9     plt.grid(True)
10
11     # Save the figure
12     plt.savefig(file_name)
13
14     # Show the figure
15     plt.show()
```

Conclusion

Thank you for your attention!